Stima della cardinalità distinta nei data stream distribuiti

Merge e compatibilità semantica degli sketch

Seminario magistrale

Daniele Ferroli

Università degli Studi di Udine

A.A. 2024–2025

Roadmap del seminario

1. Problema: perché stimare i distinti nei data stream distribuiti
2. Modello minimo: F_0, sketch, hash, mergeabilità
3. Evoluzione storica: da Probabilistic Counting a HyperLogLog++
4. Cuore del lavoro: merge, compatibilità semantica, casi `valid`/`recoverable`/`invalid`
5. Disegno sperimentale e metodologia
6. Risultati principali su streaming e merge eterogeneo
7. Conclusioni, limiti e sviluppi futuri

Il problema

- Obiettivo: stimare quanti identificatori distinti compaiono in un flusso continuo senza memorizzare tutto l'input.
- Applicazioni tipiche: monitoraggio di rete, telemetria, osservabilità, analisi di log, ottimizzazione di interrogazioni.
- Nei flussi di grandi dimensioni il conteggio esatto diventa costoso sia in memoria sia in tempo.
- In scenari distribuiti non basta stimare bene in locale: bisogna anche combinare sketch costruiti su nodi o partizioni diverse.

Messaggio guida

In questo lavoro si studia quando il merge di sketch count-distinct è semanticamente corretto, quando è recuperabile e quando va rifiutato.

Modello minimo: stream e F_0

- Il flusso è una sequenza ordinata $S = \langle x_1, x_2, \ldots, x_s \rangle$.
- In questo lavoro si considera il modello *insertion-only*: ogni aggiornamento aggiunge occorrenze, non le rimuove.
- Gli algoritmi devono elaborare il flusso in un solo passaggio, con aggiornamenti rapidi e memoria sublineare.
- La quantità d'interesse è il numero di distinti osservati.

$$F_0 = |\{a \in \mathcal{U} \mid f(a) > 0\}|$$

Interpretazione

F_0 è la cardinalità del supporto del vettore delle frequenze: coincide con il numero di elementi distinti.

Sketch, hash e mergeabilità

- Uno *sketch* mantiene una sintesi compatta del flusso e permette di stimare F_0 senza conservare tutti gli elementi.
- Negli sketch count-distinct l'hash non serve solo a randomizzare: determina come il flusso viene codificato nello stato.
- Uno sketch è *mergeabile* se combinare due stati equivale a elaborare serialmente l'unione dei due flussi, a parità di garanzie o con degradazione esplicita.

$$\hat{\theta}\big(\mathcal{K}(S_1) \oplus \mathcal{K}(S_2)\big) \approx \hat{\theta}\big(\mathcal{K}(S_1 \cdot S_2)\big)$$

$$\big(\mathcal{K}(S_1) \oplus \mathcal{K}(S_2)\big)[j] = \max\{\mathcal{K}(S_1)[j], \mathcal{K}(S_2)[j]\}$$

Punto critico

Per gli sketch HLL-like il merge a massimi è corretto solo se i due lati condividono parametri strutturali compatibili e la stessa semantica dell'hash.

Evoluzione storica degli algoritmi

Metodo	Idea dominante	Limite principale
Probabilistic Counting	Una bitmap singola osserva eventi rari nell'hash	Varianza elevata
PCSA	Più bitmap virtuali e mediazione stocastica	Stato più pesante delle famiglie a registri
LogLog	Registri + media geometrica dei massimi di rango	Sensibile alla normalizzazione e ai regimi bassi
HyperLogLog	Registri + media armonica con correzioni di intervallo	Richiede gestione di regimi diversi di cardinalità
HyperLogLog++	HLL corretto dal bias + rappresentazione sparsa/densa	Maggiore complessità ingegneristica

La linea evolutiva riduce progressivamente varianza e bias pratico mantenendo l'idea di sketch compatti e mergeabili.

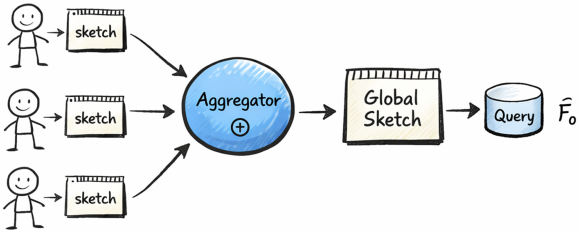
Perché HLL e HLL++ sono il riferimento pratico

Metodo	Stato dello sketch	Errore tipico	Cosa aggiunge
LogLog	Array di registri	$\approx 1.30/\sqrt{m}$	Passaggio dalle bitmap ai registri
HyperLogLog	Array di registri	$\approx 1.04/\sqrt{m}$	Stima più stabile grazie alla media armonica e alle correzioni di regime
HyperLogLog++	Registri sparsi o densi	$\Theta(1/\sqrt{m})$, con bias pratico ridotto	Correzione empirica del bias e migliore gestione delle cardinalità piccole/intermedie

- In questo lavoro HyperLogLog++ è il riferimento principale per studiare il merge distribuito.
- L'attenzione non è sull'ordine asintotico dell'errore, ma sulla stabilità empirica e sulla correttezza semantica del merge.

Perché il merge conta nei sistemi distribuiti

- Più nodi o partizioni temporali costruiscono sketch locali sullo stesso problema di cardinalità.
- Ricostruire l'insieme esatto degli elementi distinti annullerebbe il vantaggio dello sketching.
- Serve quindi un merge che preservi il significato dello stato, non solo la sua forma.
- In questo lavoro si analizzano sia il caso omogeneo sia i principali casi eterogenei.



Compatibilità semantica del merge

`valid`	`recoverable`	`invalid`
Il merge è corretto così com'è. Caso tipico: HLL++ con stessa precisione, stessa funzione di hash e stesso seed.	Il merge non è direttamente corretto, ma lo diventa dopo una normalizzazione esplicita. Caso tipico: HLL++ con precisioni diverse ma stessa semantica dell'hash.	Nel quadro di questo lavoro non esiste un merge semanticamente corretto. Caso tipico: funzioni di hash o seed diversi.

Compatibilità semantica vs strategia operativa

Compatibilità semantica

È una proprietà della coppia di sketch.
Risponde alla domanda: *esiste un merge semanticamente corretto?*
Le classi restano `valid`, `recoverable`, `invalid`.

Controllo negativo

Strategia sperimentale di merge forzato (`unsafe_naive_merge`).
Serve solo a misurare quanto degrada la stima quando si ignora la semantica.

Strategia operativa

È una scelta del sistema.
Risponde alla domanda: *dato questo caso, che cosa faccio in esecuzione?*

- Un caso `valid` ammette tipicamente `direct`.
- Un caso `recoverable` può essere `reject` oppure `reduce_then_merge`.
- Un caso `invalid` non diventa corretto se si forza il merge: in questo contesto va rifiutato.

Disegno sperimentale essenziale

- Dataset sintetico `prefix_constant_rho` con $n = 10^7$, $p = 50$ e valori controllati di

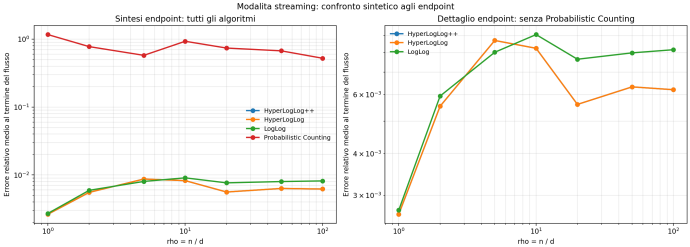
$$\rho = \frac{n}{d} \in \{100, 50, 20, 10, 5, 2, 1\}.$$

- Nella seconda vista sperimentale si osserva anche

$$\rho_t = \frac{t}{F_0(t)},$$

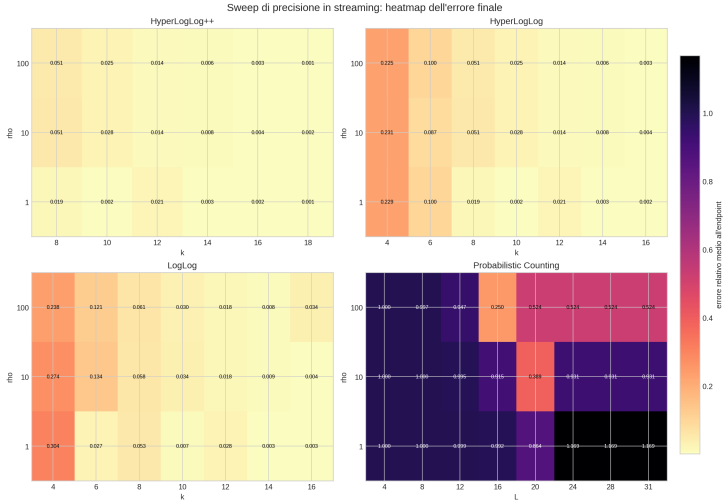
  cioè il rapporto tra elementi processati e distinti nel prefisso.
- Tre modalità di valutazione:
  - **streaming**: andamento della stima lungo i prefissi;
  - **merge omogeneo**: confronto tra merge e riferimento seriale;
  - **merge eterogeneo**: casi `valid`, `recoverable`, `invalid`.
- Metriche principali: stima media, dispersione della stima, errore relativo medio finale.
- Gli esperimenti di merge sono svolti su HyperLogLog++.

Risultato 1: comportamento degli algoritmi in streaming



- HyperLogLog++ è il metodo più regolare e accurato nel dominio osservato.
- HyperLogLog è il riferimento più vicino.
- LogLog è più sensibile ai parametri.
- Probabilistic Counting resta fuori scala rispetto agli altri.
- Nella vista su ρ_t, HLL++ scende a circa $3.95 \cdot 10^{-4}$, contro $2.08 \cdot 10^{-3}$ di HLL.

Risultato 2: l'aumento di precisione favorisce soprattutto HLL++



- HyperLogLog++ mostra l'andamento più regolare.
- Il parametro migliore osservato è sempre $k = 18$ per $\rho = 1, 10, 100$.
- HyperLogLog migliora, ma meno uniformemente.
- LogLog resta più irregolare.
- Probabilistic Counting migliora poco e resta lontano dalle famiglie a registri.

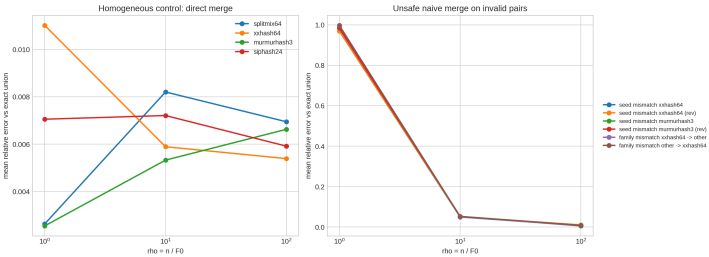
Risultato 3: il merge omogeneo è il controllo del caso `valid`

- Configurazione: HyperLogLog++, $\rho \in \{1, 10, 100\}$, $k \in \{10, 14, 18\}$, quattro funzioni di hash.
- Condizione di omogeneità: stessa funzione di hash, stesso seed e stessi parametri sui due lati.
- Totale osservato: 36 configurazioni e 900 coppie di merge.

$$\Delta_{abs} = 0, \qquad \Delta_{rel} = 0$$

ρ	Coppie totali	Coppie con delta $\neq 0$	Max Δ_{rel}
1	300	0	0
10	300	0	0
100	300	0	0

Risultato 4: hash mismatch = caso `invalid`

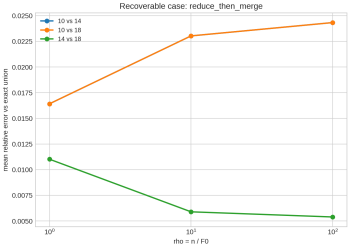


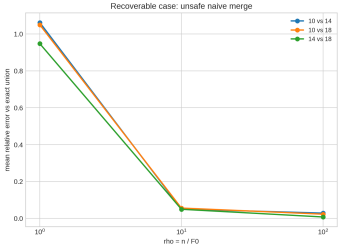
- Controllo locale omogeneo: errore medio di merge = 0.006233.
- Scenari `invalid`: errore medio di merge tra 0.342 e 0.353.
- Errore massimo osservato: 1.009634.
- Il degrado compare sia con famiglie di hash diverse sia con seed diversi.

Conclusione

Se cambia la semantica dell'hash, il merge forzato perde significato. Il caso va rifiutato.

Risultato 5: precision mismatch = caso `recoverable`





Strategia	$\overline{\varepsilon}_{merge}$
`reject`	–
`unsafe_naive_merge`	0.363882
`reduce_then_merge`	0.016651

- Il riferimento seriale è 0.016651.
- `reduce_then_merge` coincide col riferimento corretto e ha degrado medio 1.00.
- Il merge naïf ha degrado medio 32.37.

Che cosa emerge complessivamente

- Conferma sperimentalmente la linea evolutiva che porta a HyperLogLog++ come metodo più stabile nel dominio osservato.
- Mostra che il merge corretto dipende non solo dal tipo di sketch, ma anche dalla semantica condivisa dell'hash.
- Rende operativo il lessico `valid`/`recoverable`/`invalid` e lo collega a strategie di sistema ben distinte.
- Fornisce un framework riutilizzabile che separa algoritmi, hash, dataset, metriche e protocolli sperimentali.

Take-away

Negli sketch count-distinct distribuiti, hash e parametri fanno parte della semantica dello sketch: senza compatibilità semantica il merge non è un'operazione neutra.

Limiti e sviluppi futuri

Limiti del lavoro

- Dataset soprattutto sintetico e controllato
- Studio del merge condotto soprattutto a coppie
- Analisi eterogenea concentrata su hash mismatch e precision mismatch
- Parte di merge sviluppata soprattutto su HLL++

Direzioni future

- Dati reali e sorgenti di flusso operative
- Topologie distribuite più ampie
- Altri assi di eterogeneità semanticamente chiari
- Algoritmi più recenti e altre famiglie di sketch

Domande

Grazie per l'attenzione.